PortfolioSync-DSE: Dual-Adaptive Multimodal Prediction and Algorithm Selection for HLS Design Space Exploration

Lei Xu, Ruzhi Gao, Shanshan Wang, Chenglong Xiao*
Department of Computer Science, Shantou University, China
{24lxu,25rzgao,sswang,chlxiao}@stu.edu.cn

***Abstract*—High-level synthesis (HLS) automates the translation of high-level languages into RTL designs, but its vast design space requires efficient design space exploration (DSE) to balance power, performance, and area (PPA). HLS DSE typically involves two key components: QoR prediction using learning-based surrogates and search optimization via meta-heuristic or reinforcement learning algorithms. Through extensive experiments, we observed a fundamental limitation common to both components: no single QoR predictor nor any single DSE algorithm achieves consistent optimality across all benchmark kernels. For QoR prediction, no fixed combination of multimodal features yields the highest accuracy across all benchmarks, as different kernels favor different feature sets. For DSE, the No-Free-Lunch theorem implies that no single search algorithm consistently outperforms others on every problem instance; instead, different algorithms exhibit complementary strengths across benchmarks. To overcome this, we propose PortfolioSync-DSE, a dual-adaptive framework that jointly selects kernel-aware multimodal QoR predictors and the optimal DSE algorithm for each kernel, ensuring robust PPA outcomes across the entire benchmark suite. Experimental results demonstrate that trimodal fusion based on MFF (*AST+CDFG+Text*) reduces macro-averaged *All* RMSE by 13.1%–52.9% over SOTA predictors (GNN-DSE, HGBO, Ironman-Pro, and MPM). MFF-KAS further reduces *All* RMSE by 30.5% over trimodal fusion and by 39.6%–67.3% over SOTA baselines, with stronger cross-kernel robustness. Our SoberDSE framework significantly outperforms state-of-the-art heuristic-based DSE algorithms by up to 69.78% and state-of-the-art learning-based DSE methods by up to 53.60%. Furthermore, compared to conventional classification models, SoberDSE delivers superior accuracy in small-sample learning scenarios, with an average enhancement of 50.00%.**

***Index Terms*—High-level synthesis, design space exploration, algorithm selection, multimodal fusion, reinforcement learning**

I. Introduction

High-level synthesis (HLS) is a mainstream design automation methodology for implementing application-specific integrated circuit (ASIC) and field-programmable gate array (FPGA) designs. By automating the translation from behavioral specifications to register-transfer level (RTL) implementations, HLS substantially lowers the entry barrier for hardware design. Commercial HLS toolchains expose a rich set of tunable synthesis knobs, including `#pragma HLS unroll`, `#pragma HLS pipeline`, and `#pragma HLS array_partition`, to steer optimization across power-performance-area (PPA) metrics [1]. The process of optimizing HLS design space exploration (DSE) typically involves two key steps: quality of results (QoR) prediction and design space search. On one hand, performing actual HLS synthesis for each candidate configuration is prohibitively time consuming, often taking minutes to hours per run on commercial tools such as Vivado HLS or Vitis. This motivates the use of fast and accurate QoR surrogate models to estimate design metrics without invoking the full synthesis flow. On the other hand, even with an efficient predictor, the sheer size of the design space poses a severe search efficiency challenge: the number of valid pragma combinations grows exponentially with the number of directives, making exhaustive enumeration computationally intractable.

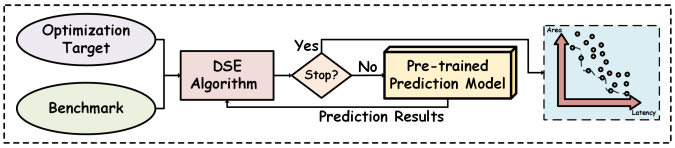


Fig. 1: Framework for HLS DSE using predictive models.

To address these challenges, a prevailing approach entails transforming high-level designs into control-dataflow graphs (CDFGs) and employing graph representation learning to extract structural features that encode intrinsic control and data dependencies embedded in source code [2]–[7]. This facilitates rapid inference of critical quality of results (QoR) metrics including latency and hardware resource utilization, eliminating the need for costly full HLS synthesis runs. Furthermore, established heuristic search algorithms [8]–[11] navigate the expansive design space to identify near-optimal pragma configurations, striking an adaptive trade-off across PPA objectives. As illustrated in Fig. 1, the DSE engine iteratively navigates the design space guided by predefined optimization targets and benchmark-specific architectural traits. It first verifies satisfaction of preset termination criteria, including maximum iteration count and convergence thresholds. Upon fulfillment, the algorithm outputs the approximated Pareto-optimal solution set. Otherwise, a pre-trained graph neural network (GNN) model infers quality of results metrics to steer subsequent search iterations, establishing a closed-loop feedback mechanism that iteratively elevates Pareto frontier quality while reducing end-to-end exploration latency relative to conventional synthesis-driven DSE pipelines. While this paradigm has demonstrated notable success, we identify several unaddressed problems as follows.

Problem 1 (Feature Information Loss): From the perspective of model learning, adopting CDFGs as input leads to the omission of several critical categories of information embedded in the original behavioral descriptions [12]. Specifically, CDFGs fail to capture the precise semantics of pragma directives, omit variable-level semantic and type details, and discard structural cues regarding loop nesting hierarchies. While augmenting

CDFGs with textual features partially recovers explicit pragma literals and surface-level semantics, this combination remains insufficient to resolve implicit structural dependencies, notably precise pragma-loop bindings and hierarchical control flow relationships, which are often obfuscated during graph construction.

Problem 2 (Cross-Benchmark Modality Redundancy): Current fusion strategies enforce uniform concatenation of features across all candidate modalities irrespective of kernel-specific architectural traits [12], [13]. Actually, we observed that no fixed modality combination generalizes optimally to all benchmark kernels: for compute-bound *stencil* kernel, abstract syntax tree (AST)-derived loop hierarchy features dominate latency prediction, while text-encoded semantic cues contribute negligible marginal utility; conversely, for memory-bound sparse matrix kernels, CDFG-extracted data dependency patterns overlap heavily with AST-based structural information. This indiscriminate feature integration inflates model complexity and undermines cross-kernel prediction robustness.

Problem 3 (Cross-Kernel Single-Algorithm Deficiency): From the perspective of multi-objective optimization, the No-Free-Lunch theorem [14] establishes that no single search algorithm can attain consistent optimality across all problem instances. Our experimental observations confirm this phenomenon in the context of HLS DSE, where the relative performance of different DSE algorithms varies notably across benchmark kernels, with no single algorithm dominating universally. Fixing a single DSE algorithm across all benchmarks therefore inevitably produces inferior Pareto frontiers and erodes end-to-end exploration efficiency.

To address the aforementioned limitations, we propose PortfolioSync-DSE, a dual-adaptive orchestration framework that jointly selects suitable kernel-aware multimodal QoR predictors and DSE algorithms. PortfolioSync-DSE extends the prior work SoberDSE [15]. Compared to SoberDSE, PortfolioSync-DSE introduces the following extensions:

- To the best of our knowledge, there is no prior work that directly integrates Abstract Syntax Tree (AST) representations into a unified multimodal learning framework for HLS QoR prediction. Existing GNN-based predictors predominantly rely on control and data flow graphs (CDFG) or pragmatic IR representations, leaving the structural semantics encapsulated in AST largely unexplored in the context of cross-modal fusion. By incorporating AST as an additional modality alongside CDFG and text pragmas, our work pioneers a more comprehensive characterization of kernel behavior.
- Through extensive experiments across 20 HLS kernels, we observe that no single modality or static modality combination consistently achieves the best prediction accuracy for all problem instances. Compute-bound kernels derive substantial benefit from structural features, whereas memory-bound kernels exhibit severe redundancy between CDFG and AST representations, and in some cases the AST+CDFG+Text combination even underperforms the CDFG+Text baseline. This finding motivates us to propose a kernel-aware prediction expert selection mechanism, namely KAS, which dynamically identifies the optimal modality combination for each kernel and for each prediction metric.
- Built upon the prior work SoberDSE, we integrate the aforementioned QoR prediction model, MFF-KAS, into a unified HLS Design Space Exploration framework named PortfolioSync-DSE. This framework jointly optimizes modality selection and DSE algorithm recommendation, achieving consistent performance improvements over both the baseline and static trimodal fusion across heterogeneous kernel architectures.

The overall workflow of PortfolioSync-DSE is shown in Fig. 3. Our main contributions are as follows:

- *MFF-KAS (Section IV-B):* We propose a Multimodal Feature Fusion framework that integrates CDFG, AST, and source-code Text through cross-attention-based fusion, and systematically evaluates all seven non-empty subsets of {CDFG, AST, Text}. We design meta-learning based kernel aware selector to adaptively select among seven pretrained experts per kernel and metric, achieving improved prediction accuracy and cross-kernel generalization over SOTA models.
- *SoberDSE (Section IV-C):* We establish the *SoberDSE* framework for DSE algorithm recommendation. we reformulate the DSE research paradigm by integrating per-benchmark algorithm recommendation, which mitigates the inherent constraints of the No-Free-Lunch theorem on fixed search strategies. To address the challenge of limited benchmark training data, we propose a hybrid supervised-reinforcement learning recommendation architecture that initializes reinforcement learning agent states with pretrained supervised model outputs, enabling high-precision algorithm selection under small-sample regimes.
- *PortfolioSync-DSE:* Each constituent module outlined above delivers standalone contributions to the fields of QoR prediction and DSE research. We unify these components into a cohesive framework termed PortfolioSync-DSE, which provides an efficient, generalizable pipeline for processing diverse real-world HLS benchmark kernels.
- Experimental results demonstrate that *MFF-KAS* reduces macro-averaged *All* RMSE by 30.5% over trimodal fusion and by 39.6% to 67.3% over SOTA predictors, with stronger cross-kernel robustness. When integrated into *SoberDSE*, it outperforms state-of-the-art heuristic-based and learning-based DSE algorithms by up to 69.78% and 53.60%, respectively, while delivering 50.00% higher accuracy in small-sample scenarios. These compelling gains motivate the unified *PortfolioSync-DSE* framework.

II. Related work and motivation

A. QoR prediction.

The evolution of QoR prediction methodologies demonstrates a clear trajectory: starting from foundational GNN-based feature extraction on CDFGs, it has progressed toward increasingly sophisticated architectures and paradigms. Early works establish GNNs as viable surrogates for HLS QoR estimation (Wu et al. [2]), with subsequent innovations enhancing topological

TABLE I
COMPARISON OF THREE FEATURE REPRESENTATIONS FOR HLS QOR PREDICTION.

Feature	Form	Information Content	Advantages	Limitations
CDFG	Graph	Basic blocks, operations, and data/control dependencies	Aligned with hardware timing and datapaths	Easily loses data types and exact pragma values
Text	Sequence	Entire source and pragma text	Complete textual content	Lacks hierarchical structure
AST	Tree	Functions, branches, loops, expressions, and pragmas	Preserves syntax and high-level semantics	Lacks explicit dataflow and control flow

awareness through edge-centric aggregation (Lin et al. [3]) and hierarchical pooling (Kuang et al. [4]). Breakthrough methodologies then emerged that bypassed traditional EDA dependencies: GNN-DSE achieved millisecond-level evaluation via surrogate modeling [5], Ferretti et al. matched industrial simulator accuracy without compiler internals [6], and Powergear pioneered direct C/C++ source processing with multi-granular embeddings [7]. Most recently, ProgSG introduced a representation learning method combining the source sequence and the CDFG [13]. MPM coupled CDFG encoders with pretrained language-model embeddings in a CDFG+Text fusion pipeline, achieving strong accuracy on MachSuite and PolyBench [12].

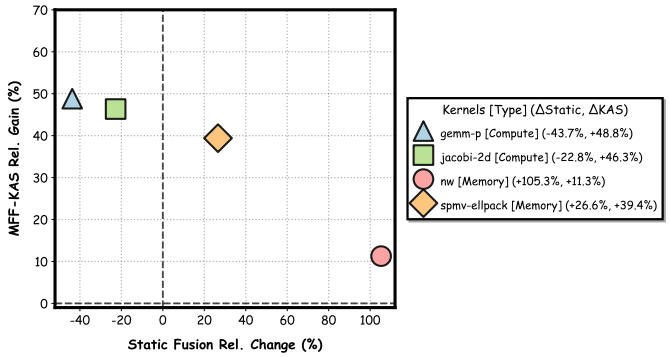
Why MFF? Existing CDFG-based learning paradigms face inherent limitations in capturing pragma semantics and structural hierarchies. While supplementing CDFGs with textual features partially mitigates this gap, critical structural information such as precise pragma-loop bindings and nested control flow relationships remains inaccessible through naive text encoding alone. As illustrated in Table I, neither standalone CDFG nor CDFG augmented with textual features can fully encompass the syntax and high-level semantics uniquely provided by ASTs. To bridge this gap, we propose MFF, which introduces the AST as a third modality to explicitly capture pragma attachments and nested loop hierarchies [16]. It thus complements CDFG views that emphasize data/control flow and Text embeddings that lack explicit hierarchical syntax. To make this distinction concrete, consider `#pragma HLS UNROLL factor=4` as an example. This directive appears in Text representations merely as a sequence of discrete tokens, making it difficult to establish structural associations with the target loop body. In CDFG, it is typically compressed into an auxiliary attribute of a loop node, which inadequately reflects the changes in parallelism and dataflow reorganization introduced by variations in the unroll factor. In contrast, within the AST, the pragma can be represented as an independent node attached directly to the corresponding `ForStmt`, where both the unroll factor and the loop-nesting hierarchy are explicitly visible. These comparisons indicate that, for HLS QoR prediction tasks requiring the preservation of both optimization directive parameters and their syntactic context, incorporating AST representations is both necessary and well justified [16].

Why KAS? Prevailing fusion strategies employ fixed feature concatenation without accounting for the varying informational utility across diverse kernel architectures. So no single modality combination generalizes across diverse kernels and QoR metrics. To address this, we develop a Kernel-Aware Selector (KAS) that dynamically routes each unseen kernel to the optimal pretrained expert among seven modality subsets, effectively eliminating the redundancy and robustness issues associated with fixed fusion policies. To validate the necessity of kernel-aware fusion, we evaluate three model configurations, namely MPM (CDFG+Text) [12], static trimodal fusion (AST+CDFG+Text), and our proposed MFF-KAS, on four representative kernels using an of per-metric root mean square error (RMSE) (latency + LUT + DSP + FF + BRAM) as the evaluation metric. As illustrated in Fig. 2(a), the x-axis quantifies the relative RMSE change of the static trimodal fusion (*AST+CDFG+Text*) against the MPM baseline (*CDFG+Text*), where positive values indicate higher error (worse performance) than MPM. The y-axis measures the relative RMSE reduction of our proposed *MFF-KAS* over MPM, with higher values denoting larger improvements. compute-bound kernels cluster in the left half of the plot, where static fusion already reduces error relative to MPM but leaves headroom for further optimization. In contrast, memory-bound kernels fall in the right "Danger Zone", where overlapping structural representations between CDFG and AST limit static fusion's effectiveness compared to the MPM baseline. Notably, all points lie above the zero horizontal line, confirming that *MFF-KAS* delivers consistent performance gains across heterogeneous kernel architectures via adaptive modality selection.

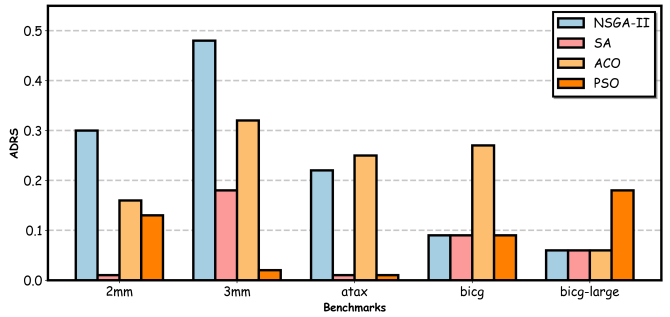
B. Design Space Exploration.

1) Heuristic-based Design Space Exploration.: Heuristic-based design space exploration has been demonstrated to effectively solve multi-objective optimization problems. Previous work utilizing heuristic algorithms such as NSGA-II [8], SA [9], and ACO [10] has achieved efficient design space exploration. To address the tendency of traditional heuristics to become trapped in local optima, Lattice-Traversing [17] integrates a Pareto-neighbor-based selection strategy with a random selection strategy, effectively balancing exploration and exploitation. To overcome the issues of slow convergence and hyperparameter sensitivity in conventional heuristics, HGBO-DSE [4] employs Bayesian optimization to effectively resolve these challenges. Furthermore, MOEDA [18] decomposes the design space exploration problem into subproblems and solves each using multi-objective evolutionary algorithms, thereby identifying a superior Pareto front.

2) Reinforcement learning-based Design Space Exploration.: The performance of heuristic-based design space exploration is constrained by the algorithms' inability to adapt to specific benchmark features and their reliance on rigid, non-adaptive search strategies. In contrast, IRONMAN-PRO [19] combines the Actor–Critic (AC) and Policy Gradient (PG) to propose the RLMD approach, which adaptively leverages benchmark features and employs flexible exploration strategies for design



(a) Modality mismatch analysis across representative kernels. The X-axis quantifies the relative change (RMSE) of static trimodal fusion (AST+CDFG+Text) over the MPM baseline (CDFG+Text), while the Y-axis measures the relative improvement (RMSE) of our proposed MFF-KAS over MPM.



(b) Actual ADRS obtained by heuristic techniques on five benchmarks.

Fig. 2: Motivation of Our Proposed *PortfolioSync-DSE*.

space exploration. This enables RLMD to discover superior Pareto fronts compared to conventional heuristic-based methods. QL-MOEA [20] enhances evolutionary algorithms by addressing their inherent limitation in autonomously selecting exploration strategies. It intelligently balances the exploration-exploitation trade-off during the search process, enabling a more thorough and efficient exploration of the design space.

Why SoberDSE? DSE, being a multi-objective optimization task involving trade-offs among conflicting metrics such as power, latency, and resource utilization, has been tackled in prior studies using a variety of methods, including heuristic algorithms and reinforcement learning (RL) based approaches. Nevertheless, consistent with the NFL theorem, none of these algorithms has been shown to consistently outperform all others across all instances. Fixing a single DSE algorithm across all benchmark kernels inevitably produces inferior Pareto frontiers and erodes end-to-end exploration efficiency. As illustrated in Fig. 2(b), we conduct DSE using four heuristic algorithms non-dominated sorting genetic algorithm II (NSGA-II) [8], simulated annealing algorithm (SA) [9], ant colony optimization (ACO) [10], and Particle Swarm Optimization (PSO) [11] on benchmarks including *2mm*, *3mm*, *atax*. *bicg* and *bicg-large* with the Average Distance to Reference Set (ADRS) as the performance metric (lower values indicate better performance). The results show that SA performs best on *2mm* and *atax*, while different algorithms excel on the remaining benchmarks. These experimental findings confirm that no DSE algorithm performs optimally across all benchmarks, underscoring the necessity of an automated system to recommend the most suitable algorithm on a case-by-case basis.

III. Preliminary

A. Kernel Characterization via Roofline Model

We classify kernels based on the Roofline model [21] using Arithmetic Intensity (AI), defined as $AI = W/Q$ (FLOPs/Byte), where W is the floating-point operation count and Q is the memory traffic in Bytes. The ridge point $RP = P_{\text{peak}}/BW_{\max}$ serves as the threshold for kernel categorization, where P_{peak} denotes the peak computational throughput of the target hardware platform measured in FLOPs/s, and $BW_{\max}$ denotes the maximum memory bandwidth measured in Bytes/s. Kernels with $AI \geq RP$ are compute-bound, while those with $AI < RP$ are memory-bound.

We evaluate four representative kernels: *gemm-p* and *jacobi-2d* are compute-bound. *gemm-p* exhibits high AI due to cubic operations versus quadratic memory access, while *jacobi-2d* benefits from data reuse via stencil patterns. Conversely, *nw* and *spmv-ellpack* are memory-bound. *nw* suffers from low reuse in dynamic programming, and *spmv-ellpack* involves indirect memory accesses with low data density.

This heterogeneity explains the varying utility of multimodal features: compute-bound kernels rely on structural features (AST), whereas memory-bound kernels suffer from redundancy between CDFG and AST features, motivating our dynamic fusion approach.

B. Multimodal Learning for HLS QoR Prediction

Consider a hardware kernel k and a configuration c within its design space, where the configuration is jointly determined by the source code and pragma settings. After HLS synthesis and implementation, each (k, c) pair produces a QoR vector $y(k, c)$, which includes performance-related metrics (Latency) as well as resource-related components such as LUT, FF, DSP, and BRAM utilization. Since design space exploration requires evaluating QoR over a large number of configurations, this work aims to learn a pre-synthesis surrogate mapping for QoR prediction [5]. Given the ground-truth QoR vector $y(k, c)$ obtained from HLS synthesis, we learn a surrogate f such that

$$\hat{y}(k, c) = f(k, c), \tag{1}$$

where $\hat{y}(k, c)$ is trained to approximate $y(k, c)$.

For the same (k, c) pair, three types of representations are constructed (as summarized in Table I). For a modality subset $\mathcal{M}$ and a QoR metric t, each expert predicts

$$\hat{y}^{(t)}_{\mathcal{M}}(k, c) = f^{(t)}_{\mathcal{M}}(\mathbf{x}_{\mathcal{M}}(k, c)), \tag{2}$$

where $\mathbf{x}_{\mathcal{M}}(k, c)$ denotes the multimodal input constructed from (k, c) under $\mathcal{M}$. When $|\mathcal{M}| = 1$, $f^{(t)}_{\mathcal{M}}$ is a single-modality regressor; when $|\mathcal{M}| \geq 2$, MFF first fuses the graph and sequence-level representations of CDFG, AST, and Text into a unified embedding, and a metric-specific head then outputs $\hat{y}^{(t)}$.

C. Algorithm Selection

The algorithm selection philosophy, widely applied in multi-objective optimization, suggests that complex algorithmic design is not the only approach to further optimization. Recommending a potentially optimal algorithm based on the characteristics of a given problem instance is evidently more efficient. For a multi-objective optimization problem $\mathcal{P}$, an algorithm selection framework can be formulated as follows [22]: given a set $\mathcal{I}$ of instances of problem $\mathcal{P}$, a set $\mathcal{A} = \{A_1, \ldots, A_n\}$ of candidate algorithms, and a target optimization metric $m : \mathcal{A} \times \mathcal{I}$ that measures the performance of any algorithm $A_i \in \mathcal{A}$ on instance set $\mathcal{I}$, the goal is to construct a algorithm selector $\mathcal{S}$ that maps each problem instance $j \in \mathcal{I}$ to an algorithm $\mathcal{S}(j) \in \mathcal{A}$ such that the overall performance of $\mathcal{S}$ on $\mathcal{I}$ is optimized according to metric m.

D. Problem Formulation

In this work, we propose to leverage the algorithm selection philosophy to identify high-performing DSE algorithms tailored to specific benchmarks. Accordingly, we frame the problem in terms of the following two subproblems:

Problem 1: Build the Algorithmic Selection Model. For an FPGA accelerator kernel (benchmark) B described using a high-level C-like language, its design space is denoted as I. This scenario involves a candidate algorithm set $\mathbf{A}$ for DSE, an optimization objective ADRS, and a pre-trained Quality of Result (QoR) prediction model PM. For the optimization objective ADRS, we leverage candidate algorithms from set $\mathbf{A}$ to explore the benchmark B. The design configurations $dc \in I$ discovered during this exploration are then evaluated using a pre-trained model P_m, which ultimately yields the performance result $\mathcal{H}(B)$ for each algorithm $\mathbf{A}_k \in \mathbf{A}$ on the benchmark B. The objective is to find an algorithmic recommendation function $\mathcal{F}$ that can approximate $\mathcal{H}(B)_{min}$: $min(\mathcal{H}(B))$ for any given benchmark B with any design space I:

$$\min_{\mathcal{F}} \left(Loss(\mathcal{H}(B)', \mathcal{H}(B)_{min}) \right) \tag{3}$$

where $\mathcal{H}(B)'$ is the approximate value generated by function $\mathcal{F}$.

Problem 2: Identify the Pareto frontier Configurations. For the benchmark B, design space I and $\mathcal{F}$ defined above, $\mathcal{F}$ recommends a potentially optimal algorithm $\mathbf{A}_{best} \in \mathbf{A}$ based on the characteristics of B to search for the Pareto frontier configurations Pf. The definition of Pareto frontier Pf is as follows:

$$a(d_{Pf}) \leq a(d_I) \ and \ l(d_{Pf}) \leq l(d_I) \tag{4}$$

where $d_{Pf} \in Pf$, $d_I \in I$ and $Pf \subseteq I$. The function *a(·)* and *l(·)* represent the resource utilization and the latency of d_{Pf} and d_I.

IV. OVERALL FRAMEWORK

Fig. 3 illustrates the overall framework of *PortfolioSync-DSE*, which mainly consists of two components: *MFF-KAS* and *SoberDSE*. During the offline training phase, we first train *MFF-KAS* to accurately estimate the QoR of a given design configuration. Specifically, QoR metrics and benchmark behavioral descriptions are used as inputs to construct the training dataset. For each benchmark, we extract multimodal representations, including CDFG, behavioral description (source code), and AST, to capture both structural and semantic information. After *MFF-KAS* converges, we proceed to train *SoberDSE*. A suite of established DSE algorithms, such as NSGA-II, Simulated Annealing, and Particle Swarm Optimization, is invoked to explore the design space of each benchmark. During this exploration, the trained *MFF-KAS* acts as a surrogate model to predict the QoR of sampled design configurations, thereby avoiding time-consuming RTL synthesis and implementation. The performance metrics of each DSE algorithm, including convergence rate and hypervolume, together with the benchmark features extracted from behavioral descriptions, form the training dataset for *SoberDSE*. This enables *SoberDSE* to learn the mapping from kernel characteristics to the most effective DSE algorithm. In the online inference workflow, *PortfolioSync-DSE* first takes a new design's behavioral description and design space definition as input. The trained *SoberDSE* analyzes these features and recommends the most suitable DSE algorithm for this specific kernel. Subsequently, the selected DSE algorithm explores the design space with the guidance of the trained *MFF-KAS*, which predicts the QoR for each generated design configuration. This collaborative pipeline ensures that both QoR estimation and algorithm selection are adapted to the unique characteristics of each benchmark.

A. Dataset Generator

Fig. 4 illustrates the dataset construction pipeline, which primarily involves the generation of three modalities: **CDFG**, **AST**, and **Text features**. The dataset construction pipeline takes behavioral descriptions and partial pragma configurations as inputs. To generate the CDFG modality, source code is first compiled to LLVM IR by a compiler front end [23], and program analysis tools such as ProGraML [24] construct a CDFG whose nodes denote LLVM IR instructions and whose edges represent data and control dependences, with pragma-related information attached during graph construction [5] so that design parameters are reflected in both the graph topology and node attributes. In parallel, an abstract syntax tree is obtained by parsing C/C++ with Tree-sitter [16] to recover structural cues such as loop nesting hierarchies and pragma-loop bindings, while the text modality is produced by concatenating source code and pragma literals, which are then tokenized and encoded by a pretrained code language model such as CodeBERT [25] to preserve global semantics. Together with actual synthesis reports that provide ground truth QoR values, these three modalities form the complete dataset for training our prediction model.

B. MFF-KAS: Multimodal Kernel-Aware Selector.

1) MFF: As depicted in Fig. 5(a), the Data Generator introduced in Section IV-A generates three modal features, uses a GNN Encoder to extract features from the CDFG and the AST, and employs a multi-head cross-attention mechanism to perform two-stage feature fusion.

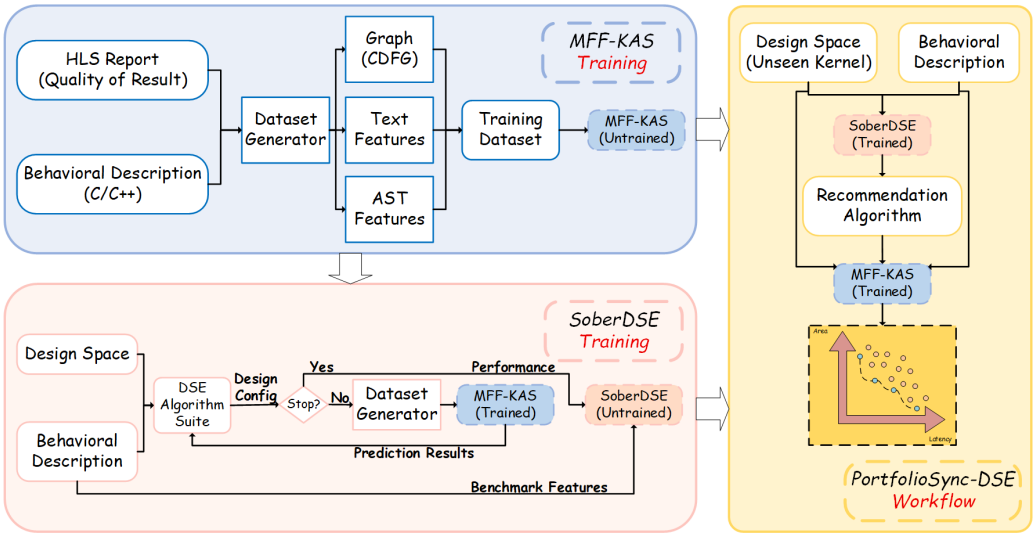


Fig. 3: *PortfolioSync-DSE* is a learning-based framework composed of *MFF-KAS* and *SoberDSE*. During the training phase, QoR metrics and benchmark behavioral descriptions are used as inputs to construct the training dataset for *MFF-KAS*. For *SoberDSE* training, a suite of DSE algorithms explores the design space of the benchmarks, while the trained *MFF-KAS* predicts the QoR of design configurations. The performance metrics of each DSE algorithm, together with the benchmark features, form the training data for *SoberDSE*. In the workflow of *PortfolioSync-DSE*, the trained *SoberDSE* first recommends a suitable DSE algorithm based on the design space and behavioral description features. Subsequently, the trained *MFF-KAS* predicts the QoR for the generated design configurations.

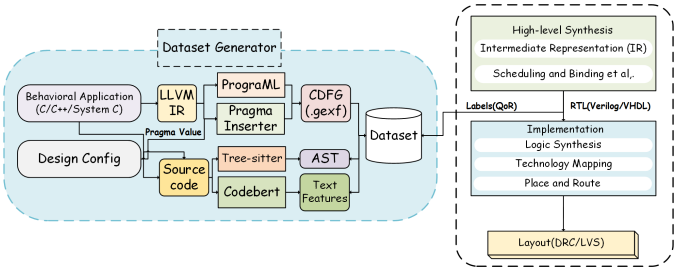
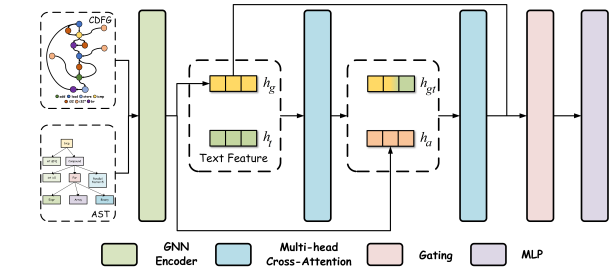
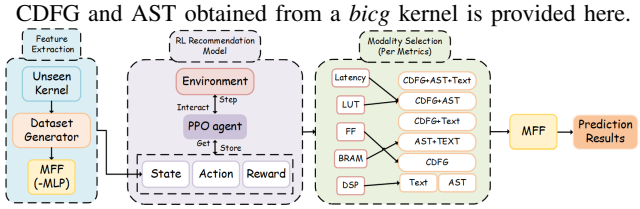


Fig. 4: Using behavioral descriptions and partial pragma configurations, we generate CDFG, AST, and text features via Programl and Tree-sitter. Together with actual synthesis reports, these form the dataset for our prediction model.



(a) MFF is a multimodal fusion framework that leverages a multi-head cross-attention mechanism to fuse CDFG (h_g), AST (h_a), and Text features (h_t) in two steps. An example of the resulting CDFG and AST obtained from a *bicg* kernel is provided here.



(b) KAS automatically selects the corresponding modality combination for each prediction metric based on the features of unseen kernels, and applies this combination within MFF for QoR prediction.

Fig. 5: The workflow of the proposed MFF-KAS framework. (a) illustrates the multi-modal feature fusion mechanism. (b) details the kernel-aware selector process.

Given $\mathbf{h}_g$ (CDFG), $\mathbf{h}_a$ (AST), and $\mathbf{h}_t$ (Text features), each expert e uses a modality subset $\mathcal{M}_e$; any disabled modality is zeroed before projection so that one fusion stack supports seven input masks. *MFF* first fuses CDFG with Text and then injects AST in two cross-attention stages.

Stage 1: Fusing CDFG and Text. We fuse graph-level CDFG with sequence-level Text before adding AST. Because $\mathbf{h}_g$, $\mathbf{h}_a$, and $\mathbf{h}_t$ have different native sizes, we linearly project each active embedding to a shared fusion dimension d before cross-attention. $\mathbf{Q}$, $\mathbf{K}$, and $\mathbf{V}$ denote the query, key, and value matrices in cross-attention. For Stage 1, $\mathbf{Q}_1$ is formed from $\mathbf{h}_g$ and $(\mathbf{K}_1, \mathbf{V}_1)$ from $\mathbf{h}_t$. With H heads,

$$\mathbf{h}_{gt} = \mathrm{MHCA}(\mathbf{Q}_1, \mathbf{K}_1, \mathbf{V}_1), \tag{5}$$

where each head uses:

$$\mathrm{Attn}(\mathbf{Q}, \mathbf{K}, \mathbf{V}) = \mathrm{softmax}\left(\frac{\mathbf{Q}\mathbf{K}^\top}{\sqrt{d_k}}\right)\mathbf{V}. \tag{6}$$

Here d_k is the per-head key dimension used for scaled dot-product attention; we set $d_k = d/H$, where d is the shared fusion dimension and H is the number of attention heads. A residual connection and layer normalization on the CDFG query branch preserve the graph anchor.

Stage 2: Incorporating AST. Using Eq.(6), $\mathbf{h}_a$ is merged into $\mathbf{h}_{gt}$ with $\mathbf{Q}_2$ from $\mathbf{h}_{gt}$ and $(\mathbf{K}_2, \mathbf{V}_2)$ from $\mathbf{h}_a$, yielding $\mathbf{h}_{\text{fuse}}$ with the same residual pattern.

Gating. Cross-attention fusion can over-emphasize Text/AST

context and under-use the CDFG embedding that carries dataflow and control structure. We therefore gate the fused vector with the projected CDFG representation:

$$\begin{aligned} \boldsymbol{\alpha} &= \sigma(\mathbf{W}_g[\mathbf{h}_g; \mathbf{h}_{\text{fuse}}]), \\ \mathbf{h}_{\text{fuse}'} &= \boldsymbol{\alpha} \odot \mathbf{h}_g + (1 - \boldsymbol{\alpha}) \odot \mathbf{h}_{\text{fuse}}. \end{aligned} \tag{7}$$

where $\boldsymbol{\alpha}$ is broadcast over feature dimensions. The gated output $\mathbf{h}_{\text{fuse}'}$ is fed into five metric-specific MLP regression heads(Latency, LUT, FF, DSP, and BRAM). The heads share the multimodal encoder and fusion trunk but use independent parameters and are optimized jointly in a single forward pass. Each modality-specific expert is thus one multi-output regressor (one model, five heads), rather than five independently trained networks.

2) KAS: As depicted in Fig. 5(b), KAS leverages MFF without its MLP prediction head to extract features from unseen kernels, integrates these features into the state of a PPO agent to dynamically interact with the environment and recommend a modality combination for each predict.

KAS selects one of seven pre-trained experts based on the multimodal kernel representation and the target QoR metric. The selector is trained via reinforcement learning, with rewards derived from validation prediction errors. We adopt reinforcement learning because expert selection must minimize downstream QoR prediction error at deployment, not merely fit best-expert labels e^*. PPO directly maximizes the performance ratio in Eq. 8, assigning higher reward to near-optimal choices and enabling exploration when several experts are similarly strong on validation. And it benefits that one-hot supervision on e^* cannot provide. We next detail its state representation, policy design, reward formulation, training procedure, and inference workflow.

State and policy. For a training or inference instance, the state s includes (i) a kernel-level context vector $\mathbf{h}_{\text{fuse}}$. (ii) an embedding of the QoR metric t. Metric-specific policy networks map s to logits over $\{0, \ldots, 6\}$, parameterizing $\pi_\theta(a \mid s)$.

Reward and PPO update. The policy is optimized with proximal policy optimization (PPO) [26]. During RL training, an action a selects expert e_a. Let $e_m^{(n)}(k, c)$ be the validation error of expert m on metric n. The immediate reward compares the chosen expert to the best available expert:

$$r(s, a) = \min_m \frac{e_m^{(n)}(k, c) + \epsilon}{e_a^{(n)}(k, c) + \epsilon}, \quad r \leftarrow \min(r, r_{\max}), \tag{8}$$

where ϵ avoids division by zero and $r_{\max}$ caps extreme ratios. Rewards are normalized within a mini-batch to form advantages. PPO maximizes a clipped surrogate objective with an entropy bonus to stabilize exploration.

Training. *KAS* policies are trained separately for each QoR metric using offline best-expert labels e^*. During PPO updates, actions closer to the e^* expert receive higher rewards (Eq. 8). After convergence, one policy checkpoint is retained per QoR metric.

Inference. Given a new kernel and QoR metric t, the trained policy returns $\hat{e}(k, t) = \arg\max_a \pi_\theta(a \mid s)$. Only $f_{\mathcal{M}(\hat{e})}^{(n)}$ is executed to produce $\hat{y}^{(n)}$ (Eq.(4)). The online cost per metric is one policy forward pass plus one expert forward pass, rather than all seven regressors. KAS selects distinct modality combinations for different kernels, aligning with Problem 2 discussed in Section II. Specifically, compute-bound kernels prioritize structural features, whereas memory-bound kernels exhibit a preference for AST+Text or CDFG+Text combinations. Different metrics may select different experts for the same kernel, reflecting heterogeneous optimality of modality combinations across kernels and QoR dimensions.

C. SoberDSE: DSE Algorithm Selector.

As illustrated in Fig. 6, our proposed *SoberDSE* framework begins with the generation of training and inference data via the Dataset Generator (Ⓐ). We first implement a DSE algorithm suite comprising seven heuristic algorithms and three reinforcement learning methods. During DSE execution, the pre-trained *MFF-KAS* leverages kernel-specific features to recommend optimal modality combinations and subsequently predicts the QoR for each design configuration. Benchmarks are divided into training and inference sets, and the performance of all DSE algorithms on each benchmark is recorded to identify the optimal algorithm for each benchmark. The Benchmark Feature Graph Generator (Ⓑ) then constructs feature graphs for each benchmark. The supervised recommendation model (Ⓒ) is initially trained and used to initialize the agent's state in the RL recommendation model (Ⓓ) by estimating algorithm selection probabilities. The agent interacts with the environment based on ground-truth optimal algorithm IDs, learning from experiences stored in a buffer. During Inference (Ⓔ), *SoberDSE* is evaluated on the inference dataset generated by the Dataset Generator (Ⓐ). The pre-trained supervised and RL recommendation models (Ⓑ and Ⓒ) collaborate to suggest algorithms, which are executed on corresponding benchmarks to identify improved Pareto fronts.

1) Dataset Generator: To construct the dataset for DSE algorithm recommendation, as shown in Part (Ⓐ) of Fig. 6, we first implement a DSE algorithm suite consisting of seven heuristic algorithms (NSGA-II, SA, ACO, PSO, Lattice, HGBO-DSE, and MOEDA) and three RL algorithms (AC, PG, and QL-MOEA). These algorithms are executed on twenty benchmarks listed in Table II, with the explored design configurations evaluated using a pre-trained ECoGNN [27] model for QoR prediction. The benchmarks used for training ECoGNN are entirely separate from those employed for training the DSE algorithm recommendation model. To evaluate the performance of DSE algorithms, we adopt ADRS as the evaluation metric:

$$ADRS(\Gamma, \Omega) = \frac{1}{|\Gamma|} \sum_{\lambda \in \Gamma} \min_{\mu \in \Omega} f(\lambda, \mu) \tag{9}$$

where Γ represents the reference Pareto-optimal set, Ω denotes the approximate Pareto-optimal set, and the function f computes the distance between λ and μ. We select the solution sets obtained by 10 algorithms from the DSE algorithm suite on each kernel, compare them kernel by kernel based on the dominance relationship, and use the comparison results as the reference set. Finally, we identify the algorithm ID achieving optimal performance (lowest ADRS value) for each benchmark, which serves as the training and inference target for the subsequent recommendation model.

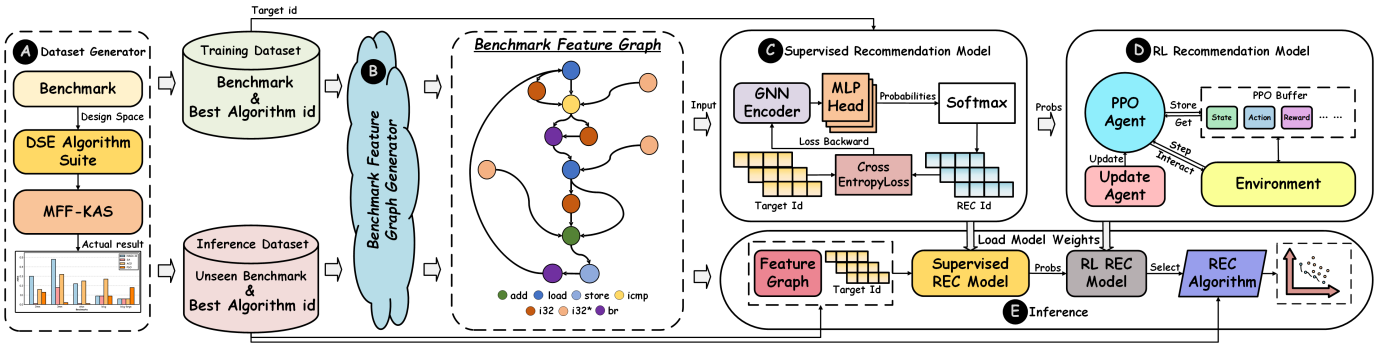


Fig. 6: The Workflow of Our Proposed ***SoberDSE***.

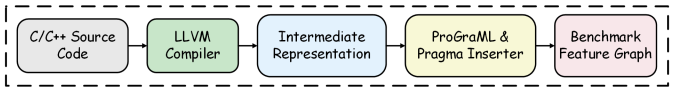


Fig. 7: Benchmark Feature Graph Generation Process.

2) Benchmark Feature Graph Generator: As discussed in Sections III-C and III-D, while the recommendation model updates its parameters and biases based on training data to select algorithm IDs, the representation of benchmarks typically in high-level languages makes it challenging to fully capture their characteristic features, which is crucial for the accuracy of the recommendation algorithm. To address this issue, we propose the Benchmark Features Graph Generator (BFGG) (Ⓑ), which represent the benchmark in the form of feature graph (BFG) by using the method of generating CDFG. As illustrated in Fig. 7, the source code from benchmark is first converted to an intermediate representation (IR) using LLVM, followed by the generation of a BFG through ProGraML [28], which incorporates HLS directives for enhanced semantic representation. Concurrently, following the GNN-DSE [5], pragma inserter constructs icmp nodes to augment the BFG with pragma-specific information.

3) Collaborative Recommendation Model: Due to the reconfigurable hardware nature of FPGA accelerator kernels (benchmarks), the number of real-world benchmarks is inherently limited, which consequently restricts the number of benchmarks available for our use. As a result, the data samples obtained from the Dataset Generator (Ⓐ) are constrained in quantity. Treating algorithm recommendation solely as a classification problem using supervised learning may lead to overfitting and poor generalization. An alternative approach involves reinforcement learning, where an agent continuously interacts with the environment to recommend optimal algorithms. However, under limited sample constraints, this method still suffers from low sample efficiency and local optima convergence. To address these issues, we propose a collaborative recommendation paradigm where algorithm IDs generated by a pre-trained supervised model initialize the RL agent's state, effectively mitigating the aforementioned limitations.

As shown in Supervised Recommendation Model (Ⓒ) of Fig. 6, we first encode the BFG provided in BFGG (Ⓑ) using a pre-trained GNN encoder [27] to obtain the graph representation $h_{\mathcal{G}}$. The GNN encoder employed here corresponds to the encoder component of the QoR prediction model. This encoding process can be formulated as:

$$h_{\mathcal{G}} = GNN_{encoder}(BFG) \tag{10}$$

Then, we set up an MLP head to decode $h_{\mathcal{G}}$ and add a softmax layer to output the recommendation probability for each algorithm. This process can be formalized as:

$$p(A|BFG) = \text{softmax}(W_2 \cdot \text{ReLU}(W_1 h_{\mathcal{G}} + b_1) + b_2) \tag{11}$$

where W_1, W_2, b_1 and b_2 are the parameter of MLP head, A is the set of DSE algorithms. We employ the Cross-Entropy loss [29] to compute the discrepancy between the algorithm ID recommended by the supervised model and the ground-truth best-performing algorithm ID during training.

After a brief training period, we save the parameters of the supervised model and transmit its final output recommendation probabilities p_{sup} to the RL recommendation model. Since the limited sample size, we employ the PPO algorithm [26] as our reinforcement learning approach due to its high sample efficiency and training stability. As shown in RL Recommendation Model (Ⓓ) of Fig. 6, The PPO agent operates in the environment where the state representation $s_q = [h_G; p_{sup}]$ combines graph embeddings $h_{\mathcal{G}}$ from the GNN encoder and prior probabilities p_{sup} from the supervised model. To enable interaction with the environment, the PPO agent incorporates two trainable neural networks: an actor network and a critic network. The actor network selects algorithm $a_q \in A$ based on the current state s_q:

$$\pi_\gamma(a_q|s_q) = \text{softmax}(\text{MLP}_a(s_q)) \tag{12}$$

where MLP_a is used to approximate the probability distribution of algorithm recommendations $p(a_q|s_q)$. The critic network estimates state values for advantage calculation:

$$V_\phi(s_q) = \text{MLP}_c(h_G) \tag{13}$$

where MLP_a is used to calculate state values. The environment provides rewards r_q based on performance deviation:

$$r_t = \frac{|m^{(a_q)} - \bar{m}|}{\bar{m}} \tag{14}$$

where $m^{(a_q)}$ represents the ADRS value of algorithm a_q and $\bar{m}$ denotes the ADRS value of the best-performing algorithm on the corresponding benchmark. The PPO buffer serves as a critical component in the training pipeline by facilitating the interaction between the agent and environment while enabling stable policy updates.

TABLE II
THE TRAINING BENCHMARK USED FOR DSE ALGORITHM RECOMMENDATION.

Benchmarks	Functions	|DS|
	MachSuite [31]	
gemm-blocked	O(n3) algorithm for dense matrix multiplication.	2,314
gemm-ncubed	Blocked version of matrix multiplication.	7,152
	Polyhedral [32]	
atax	Matrix Transpose and Vector Multiplication.	2,644
bicg	BiCG Sub Kernel of BiCGStab Linear Solver.	1054
bicg-large	BiCG Sub Kernel of BiCGStab Linear Solver.	2,975
covariance	Covariance Computation.	970,967,172
fdtd-2d	2-D Finite Different Time Domain Kernel.	18,329,692,529
fdtd-2d-large	2-D Finite Different Time Domain Kernel.	70,303,226,437
gemm-p	Matrix-multiply.	409,905
gemm-p-large	Matrix-multiply.	690,641
gemver	Vector Multiplication and Matrix Addition.	100,977,795,072
gesummv	Scalar, Vector and Matrix Multiplication.	1,543
heat-3d	Heat equation over 3D data domain.	71,511
jacobi-1d	1-D Jacobi stencil computation.	2,871
mvt-medium	Matrix Vector Product and Transpose.	933,156
syr2k	Symmetric rank-2k update.	340,234
symm	Symmetric matrix-multiply.	50,265
trmm	Triangular matrix-multiply.	43,463
trmm-opt	Triangular matrix-multiply.	50,265
2mm	2 Matrix Multiplications	492,787,501
3mm	3 Matrix Multiplications	17,658,216,447,701

During each episode, the agent interacts with the environment by selecting actions a_q based on the current state s_q, receiving rewards r_q, and observing state transitions. These experiences $(s_q, a_q, r_q, s_{q+1}, log(\pi_\theta(a_q|s_q)), V_\phi(s_q))$ are stored in the PPO buffer. After episode completion, the buffer provides the complete trajectory data for policy optimization. The update process involves computing advantages using Generalized advantage estimation (GAE) [30]:

$$A_q = \sum_{l=0}^{\infty} (\gamma\lambda)^l \delta_{q+l} \tag{15}$$

where $\delta_q = r_q + \gamma V(s_{q+1}) - V(s_q)$, γ, λ are hyperparameters of GAE. The agent is then updated by minimizing the PPO objective:

$$\mathcal{L}_{PPO} = \mathcal{L}_{policy} + c_1\mathcal{L}_{value} - c_2\mathcal{L}_{entropy} \tag{16}$$

where $\mathcal{L}_{policy}$ is the clipped policy gradient loss that encourages policy improvement while constraining the update size to prevent drastic policy changes, $\mathcal{L}_{value}$ is the value function loss that trains the critic network to accurately estimate state values, $\mathcal{L}_{entropy}$ is the entropy bonus that encourages exploration by increasing the randomness of the policy and c_1, c_2 are hyperparameters.

By training the PPO agent and saving its parameters, we evaluate the proposed recommendationl model as shown in the Inference (❺) section of Fig. 6. First, the benchmark features graph from the inference dataset is input to the pre-trained supervised recommendation model to generate an initial algorithm recommendation probability. The RL recommendation model then integrates this probability to suggest the potentially optimal DSE algorithm and returns the Pareto front obtained by that algorithm on the corresponding benchmark.

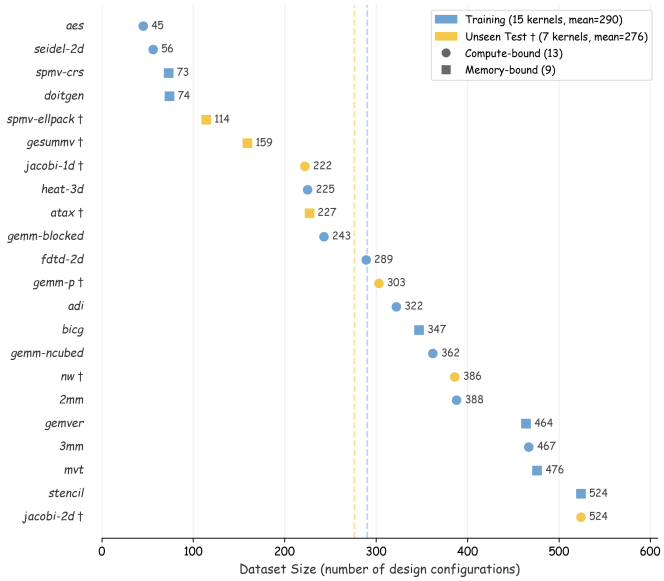


Fig. 8: Twenty-two kernels from MachSuite and PolyBench (†: seven unseen test kernels). The training pool contains 15 kernels with 4,355 design configurations in total. The seven † kernels are reserved for final evaluation only and contain 1,935 design configurations in total. Furthermore, the 20 kernels are classified into compute-bound and memory-bound categories following the approach in Section III-A. Circular and square markers represent compute-bound and memory-bound kernels, respectively. *Dataset Size* is the number of design configurations with ground truth QoR labels.

V. EXPERIMENTAL EVALUATION

A. Experimental Setup

This section aims to evaluate our proposed MFF-KAS and *SoberDSE*, and the experimental setup is divided into two parts:

1) MFF-KAS: This section evaluates the QoR prediction accuracy of *MFF-KAS* using a consistent HLS experimental setup and fixed train/test data splitting strategy across all comparisons. We aim to answer three key questions:

1) Can integrating AST into multimodal representations significantly improve HLS QoR prediction over existing predictors and unimodal baselines?
2) How do different modality combinations perform across diverse kernels and QoR metrics, and what are their individual contributions?
3) Given that no single modality fusion is universally optimal, can the proposed kernel-aware selector (KAS) consistently improve prediction accuracy and generalization across unseen kernels?

We use 22 kernels from MachSuite [31] and PolyBench [32], listed in Fig. 8. Each design configuration uses three inputs: a ProGraML-based CDFG [24], a program AST, and tokenized source text [25]. Ground-truth QoR labels are obtained by synthesizing each kernels with its pragma settings on AMD UltraScale+ MPSoC ZCU104 using Vitis HLS and Vivado 2022.1.

We first fix a random seed and select seven kernels as unseen test benchmarks († in Fig. 8): *atax*, *gemm-p*, *gesummv*,

TABLE III
THE INFERENCE BENCHMARK USED FOR DSE ALGORITHM RECOMMENDATION.

Benchmarks	Functions	|DS|
MachSuite [31]		
nw	A dynamic programming algorithm.	15,288
stencil	A two-dimensional stencil computation.	7,591
Polyhedral [32]		
atax-medium	Matrix Transpose and Vector Multiplication.	9,454
bicg-medium	BiCG Sub Kernel of BiCGStab Linear Solver.	19,536
correlation	Correlation Computation.	513,641,633,988
jacobi-2d	2-D Jacobi stencil computation.	7,609,187
mvt	Matrix Vector Product and Transpose.	2,385,796
symm	Symmetric matrix-multiply.	50,265
symm-opt	Symmetric matrix-multiply.	730,665
syrk	Symmetric rank-k update.	340,234

jacobi-1d, *jacobi-2d*, *nw*, and *spmv-ellpack*. The remaining 15 kernels contain 4,355 design configurations in total. These configurations are randomly split into training and validation sets with a ratio of 8:2. In addition, we categorize the 20 kernels according to the methodology described in Section III-A into two classes: compute-bound and memory-bound. In the figure, circular markers denote compute-bound kernels, while square markers denote memory-bound kernels. All regressors are trained on the training split and tuned on the validation split. *KAS* supervision labels are generated from the validation split. Final evaluation is conducted on all design configurations of the seven unseen kernels. We adopt root mean square error (RMSE) as the primary evaluation metric, as it is a standard choice in HLS QoR prediction and enables consistent error comparison across Latency, LUT, DSP, FF, and BRAM despite their different scales [5]. For each QoR metric, we compute the RMSE on every unseen kernel and then report the unweighted mean over the seven kernels (macro average), ensuring that each test kernel contributes equally. Detailed per metric and per kernel results are presented in the following subsections.

We compared four state-of-the-art (SOTA) QoR predictors, namely GNN-DSE [5], HGBO [4], Ironman-Pro [19], and MPM [12], as well as four *MFF* variants incorporating AST features: AST-only, AST+CDFG, AST+Text, and AST+CDFG+Text. During inference, it performs kernel-level expert selection among the seven pre-trained fusion experts. Seven fusion experts (AST-only, CDFG-only, Text-only, AST+CDFG, AST+Text, CDFG+Text, and AST+CDFG+Text) are pretrained independently under the *MFF* architecture using the same data partition. Expert checkpoints are frozen before KAS training; at inference, only the selected expert is executed, and fusion weights are not retrained on the test set. The seven fusion experts and KAS share the same optimizer, learning rate, training epochs, and early-stopping rule on the validation set.

2) SoberDSE: As outlined in Section IV-A, we construct datasets for training and evaluating the proposed *SoberDS*E framework. The training set comprises 20 benchmarks listed in Table II. As shown in Table III, the inference dataset comprises 10 benchmarks sourced from MachSuite and Polyhedral, exhibiting significant diversity in design space scale including the hundred-billion-level benchmark (*correlation*). The benchmarks for algorithm recommendation are completely separate from those used for QoR prediction model training. Additionally, the main methods we compare include seven heuristic algorithms (NSGA-II [8], SA [9], ACO [10], PSO [11], Lattice [17], HGBO-DSE [4], and MOEDA [18]) and three reinforcement learning algorithms (IRONMAN-PRO [19] with AC and PG, and QL-MOEA [20]). The supervised model employs a hidden layer dimension of 256, a learning rate of 0.0003, and 250 training epochs, while the reinforcement learning model uses a hidden layer dimension of 256, a learning rate of 0.0005 and 1000 training epochs.

TABLE IV
RMSE OF MPM, SOTA MODELS, AND MFF FUSION VARIANTS ON UNSEEN TEST KERNELS (MACRO-AVERAGED PER QOR METRIC). *All* IS THE SUM OF RMSE OVER LATENCY, LUT, DSP, FF, AND BRAM.

Method	Latency	LUT	DSP	FF	BRAM	All
GNN-DSE [5]	0.692	0.109	0.145	0.083	0.058	1.086
HGBO [4]	0.492	0.082	0.057	0.047	0.069	0.747
Ironman-Pro [19]	0.702	0.038	0.046	0.033	0.043	0.862
MPM [12]	0.459	**0.034**	**0.040**	**0.021**	0.034	0.588
AST-only	0.382	0.076	0.089	0.067	0.075	0.688
AST+CDFG	0.361	0.046	0.064	0.058	0.055	0.584
AST+Text	0.442	0.036	0.049	0.036	**0.032**	0.594
AST+CDFG+Text	**0.297**	0.067	0.058	0.042	0.047	**0.511**

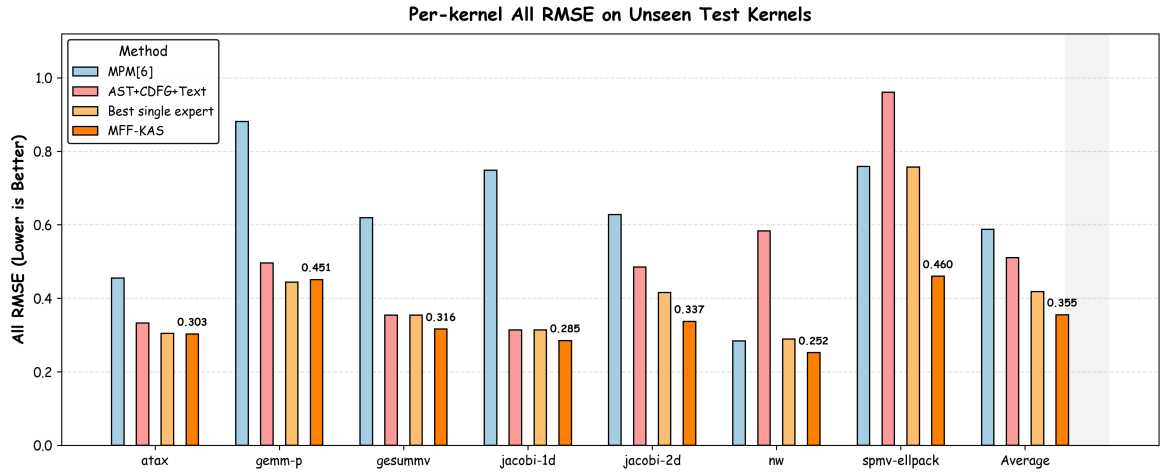
TABLE V
RMSE LOSS OF SEVEN FUSION EXPERTS ON UNSEEN KERNELS.

Fusion expert	Latency	LUT	DSP	FF	BRAM	All
AST-only	0.382	0.076	0.089	0.067	0.075	0.688
CDFG-only	0.418	0.081	0.086	0.064	0.095	0.744
Text-only	0.492	**0.035**	**0.046**	**0.031**	**0.031**	0.635
AST+CDFG	0.361	0.046	0.064	0.058	0.055	0.584
AST+Text	0.442	0.036	0.049	0.036	0.032	0.594
CDFG+Text	0.453	0.050	0.052	0.042	0.041	0.638
AST+CDFG+Text	**0.297**	0.067	0.058	0.042	0.047	**0.511**

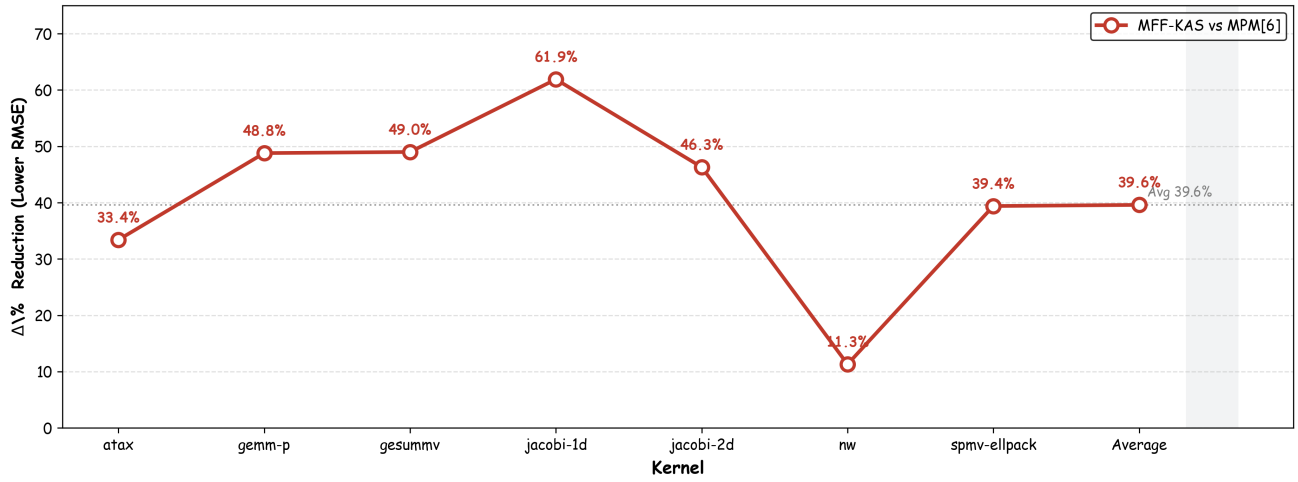
B. Evaluation of MFF-KAS

1) Comparison with SOTA models: Table IV reports macro-averaged RMSE on the seven unseen test kernels († in Fig. 8). The upper four rows are prior learning-based QoR predictors; the lower four rows are *MFF* variants that incorporate AST. Among SOTA baselines, MPM [12] achieves the lowest *All* RMSE (0.588) and serves as our primary reference.

Adding AST consistently helps: AST-only (0.382 Latency) already beats all four baselines on Latency. AST+CDFG further improves Latency and *All* RMSE to 0.361 and 0.584; AST+Text is strongest on BRAM (0.032) but weaker on Latency than AST+CDFG. AST+CDFG+Text attains the best Latency (0.297) and *All* RMSE (0.511), i.e., 13.1% lower *All* than MPM and 25.7% lower than AST-only. Still, MPM and AST+Text win on several resource metrics, so even the best fixed trimodal fusion is not uniformly optimal across QoR metrics.



(a) Per-kernel All RMSE on unseen test kernels for MPM, fixed trimodal fusion (AST+CDFG+Text), the best single expert per kernel, and MFF-KAS (lower is better). All is the sum of RMSE over Latency, LUT, DSP, FF, and BRAM.



(b) $\Delta\%$: relative All-RMSE reduction of MFF-KAS over MPM. Each point corresponds to one kernel, showing the percentage decrease in total RMSE achieved by the proposed method.

Fig. 9: Evaluation of multiple methods on unseen test kernels.

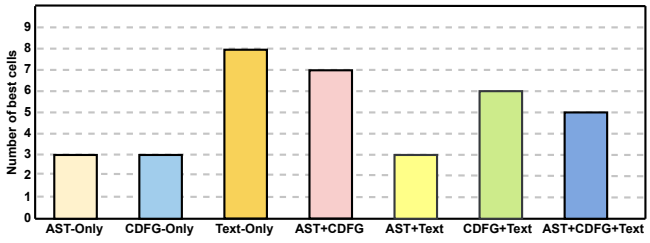


Fig. 10: Best fusion expert across unseen test kernels and QoR metrics. Each bar shows how many of the 35 $(\text{kernel}, \text{metric})$ pairs (seven kernels $\times$ five metrics) achieve the lowest RMSE with that expert.

2) Evaluation of All Modality Subsets: Table V reports RMSE for seven *MFF* experts trained as in Section V-B1 on the seven unseen test kernels. *AST-only* attains the lowest unimodal Latency RMSE (0.382), ahead of *CDFG-only* (0.418) and *Text-only* (0.492). *Text-only* yields the best RMSE on all four resource metrics (LUT, DSP, FF, and BRAM). *CDFG-only* records the highest unimodal *All* RMSE (0.744).

TABLE VI
MACRO-AVERAGED RMSE ON UNSEEN KERNELS: *AST+CDFG+Text* VS. *MFF-KAS*.

Method	Latency	LUT	DSP	FF	BRAM	All
AST+CDFG+Text	0.297	0.067	0.058	0.042	0.047	0.511
MFF-KAS	**0.223**	**0.033**	**0.040**	**0.028**	**0.031**	**0.355**

Among bimodal experts, *AST+CDFG* achieves the second-best Latency (0.361) and a low *All* RMSE (0.584). *AST+Text* preserves strong BRAM accuracy (0.032) but degrades Latency to 0.442. *CDFG+Text* lowers *All* RMSE to 0.638 compared with *CDFG-only*, yet its Latency (0.453) remains higher than *CDFG-only*, and its resource RMSE is worse than *Text-only* on every utilization column.

AST+CDFG+Text achieves the best Latency (0.297) and best *All* RMSE (0.511), but it is not the per-column winner on resource metrics, where *Text-only* or *AST+Text* remain superior.

3) No Single Fusion Expert Fits All Kernels and Metrics: The aforementioned results show that although

AST+CDFG+Text achieves the best Latency (0.297) and the best overall RMSE (0.511) among the seven fusion strategies, it does not consistently outperform on every resource metric (LUT, DSP, FF, BRAM) for every kernel. Fig. 11 further visualizes, for each unseen kernel and each QoR metric, which of the seven pretrained experts attains the lowest RMSE. The height of each bar indicates the number of winning (kernel, metric) pairs out of a total of 35. Notably, *Text-only* is optimal in eight cells, followed by *AST+CDFG* in seven and *CDFG+Text* in six, whereas *AST+CDFG+Text* is optimal in only five cells, despite having the best macro averaged Latency and overall RMSE in Tables IV and V. For instance, on the *nw* kernel, *AST+Text* yields the best Latency, while *Text-only* performs best on LUT and BRAM; on *spmv-ellpack*, *AST-only* achieves the best Latency but falls short on the four resource metrics.

4) Evaluation of Kernel-Aware Expert Selection: Given the heterogeneity observed in Fig. 11, we evaluate *MFF-KAS*, which dynamically routes each unseen test kernel to one of the pretrained fusion experts at inference time. We compare four strategies: (i) MPM: the best performing SOTA model. (ii) *AST+CDFG+Text*: the trimodal fusion expert applied uniformly to all unseen kernels. (iii) Best single expert per kernel: for each kernel, the expert with the lowest overall RMSE across seven fusion variants is selected and used for all five QoR metrics. (iv) *MFF-KAS*: our proposed method.

Macro-average results: Table VI summarizes macro-averaged RMSE on the seven unseen test kernels. *MFF-KAS* reduces RMSE on every QoR metric relative to *AST+CDFG+Text*. *All* RMSE drops from 0.511 to 0.355 (30.5% relative reduction); Latency RMSE drops from 0.297 to 0.223 (24.9%). *MFF-KAS* also outperforms every fusion experts listed in Table V on all five metrics.

Per-kernel results: Fig. 9 compares per-kernel *All* RMSE on the seven unseen test kernels for MPM [12], fixed trimodal fusion (*AST+CDFG+Text*), the best single expert per kernel, and *MFF-KAS*. The Δ% column reports the relative *All*-RMSE reduction of *MFF-KAS* over MPM. Compared with MPM, *MFF-KAS* lowers the macro-averaged *All* RMSE from 0.588 to 0.355 (Δ% = 39.6%) and improves every kernel, with per-kernel gains ranging from 11.3% (*nw*) to 61.9% (*jacobi-1d*). The largest reductions appear on *jacobi-1d*, *gemm-p*, *gesummv*, and *spmv-ellpack*. Relative to fixed trimodal fusion, *MFF-KAS* also reduces macro-averaged *All* RMSE from 0.511 to 0.355 (30.5%).

Furthermore, as stated in *Problem* 2 of Section I, the modality information required for prediction varies across kernels depending on their architectural type. Each kernel can be categorized as either compute-bound or memory-bound. As illustrated in Fig. 9, the compute-bound group includes *gemm-p*, *jacobi-1d*, *jacobi-2d*, and *nw*, while the memory-bound group comprises *atax, gesummv, and spmv-ellpack.* The MPM baseline exhibits higher prediction errors on compute-bound kernels because it lacks the structural information provided by the AST modality. MPM also underperforms on memory-bound kernels. Although AST and CDFG features contain partially conflicting information, the inclusion of AST still supplies additional cues that can be leveraged for prediction. However, the static AST+CDFG+Text configuration yields even worse results than MPM on *spmv-ellpack*, which demonstrates that the information fusion between AST and CDFG introduces genuine conflicts rather than complementary signals. In contrast, MFF-KAS dynamically selects the optimal modality combination for each kernel. Consequently, its prediction error is substantially lower than both MPM and the static three-modality fusion across all kernel types.

Our experiments yield three key findings aligned with the research questions. First, integrating AST into multimodal representations significantly outperforms state-of-the-art predictors and unimodal/bimodal variants. The AST+CDFG+Text fusion achieves the lowest macro-averaged RMSE for latency and overall QoR on unseen kernels (Tables IV and V). Second, the optimal modality combination varies across kernels and metrics (Fig. 11). Text-only performs best in eight of 35 kernel-metric pairs, while AST+CDFG+Text is optimal in only five, confirming no single fusion strategy is universally superior. Third, *MFF-KAS* reduces the macro-averaged overall RMSE from 0.511 to 0.355 (30.5%) over the fixed AST+CDFG+Text baseline, with consistent improvements across all seven unseen kernels (Table VI and Fig. 9(b)).

C. Evaluation of SoberDSE

1) Recommendation Accuracy Analysis: We utilize the trained SoberDSE ((Ⓒ) and (Ⓓ)) to perform algorithm recommendations on the inference benchmarks following the Inference (Ⓔ) workflow illustrated in Figure 6. To validate the Recommendation accuracy (percentage of correct recommendations relative to the total number of samples) of our proposed algorithm, we conduct comparative evaluations of SoberDSE and against classification methods including Random Forest (RF) [33], XGBoost [34], K-NN [35], and SVM [36]. Furthermore, to validate the importance of the PPO component, we conduct experiments using only the Supervised recommendation model (SFM) for algorithm recommendation and recorded the results. As shown in Table **??**, our proposed SoberDSE recommendation model demonstrates significantly more accurate algorithm recommendations compared to traditional classification models, achieving an average improvement of 50%. In contrast, the SFM without the PPO component shows comparable accuracy to baseline models, indicating the limited performance of supervised models on small-sample problems and their susceptibility to overfitting. SoberDSE effectively addresses the overfitting issue in supervised learning by integrating reinforcement learning. Additionally, by initializing the RL recommendation model's state with probabilities from the supervised model, SoberDSE enhances the sample efficiency of the RL recommendation process.

2) DSE Performance Analysis: We evaluate the algorithms recommended by SoberDSE ((Ⓒ) and (Ⓓ)) on the benchmarks listed in Table III using the ADRS metric, comparing primarily with heuristic techniques including NSGA-II [8], SA [9], ACO [10], PSO [11], Lattice [17], HGBO-DSE [4], and MOEDA [18]. Since our approach involves algorithm recommendation followed by executing the recommended algorithm on the corresponding benchmark, the results may coincide with one of the comparative algorithms. However, due to the stochastic

TABLE VII
COMPARISON OF ADRS RESULTS BETWEEN *SoberDSE* AND SOTA HEURISTIC ALGORITHMS FOR DSE.

Benchmarks	**NSGA-II [8]**	**SA [9]**	**ACO [10]**	**PSO [11]**	**Lattice [17]**	**HGBO-DSE [4]**	**MOEDA [18]**	**SoberDSE(Our Work)**
stencil	0.0	0.0	0.0	0.0	0.0	0.0	0.0	**0.0**
nw	0.0	0.0	0.0	0.0	0.0	0.0	0.0	**0.0**
jacobi-2d	0.0376	0.0372	0.0629	0.0372	0.0372	0.0372	0.0372	**0.0372**
mvt	0.0	0.0	0.0	0.0	0.0	0.0	0.0	**0.0**
symm	0.2333	0.2521	**0.0521**	0.2431	0.1339	0.1653	0.2311	**0.0412**
symm-opt	0.0	0.0	0.0	0.0	0.0	0.0	0.0	**0.0**
syrk	0.0	0.0	0.0	0.0	0.0	0.0	0.0	**0.0**
correlation	0.0216	0.0244	0.2307	0.0165	0.1755	0.0231	0.0185	**0.0058**
atax-medium	0.0	0.0	0.0	0.0	0.0	0.0	0.0	**0.0**
bicg-medium	0.0	0.0	0.0	0.0	0.0	0.0	0.0	**0.0**
Avg	0.0139	0.0223	0.0207	0.0293	0.0314	0.0346	0.0297	**0.0084**
SoberDSE Improv. over Lattice					**75.79%**			
SoberDSE Improv. over HGBO-DSE					**62.83%**			
SoberDSE Improv. over MOEDA					**70.73%**			

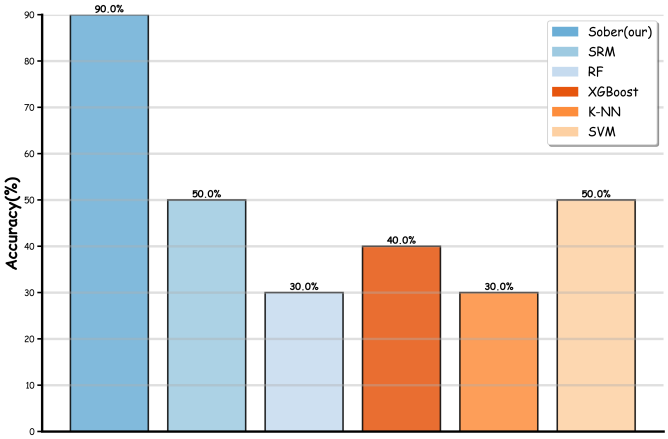


Fig. 11: Comparing Recommendation Accuracy of *Sober*, SFM and Baseline Models.

TABLE VIII
COMPARISON OF ADRS RESULTS BETWEEN *SoberDSE* AND SOTA DSE RL ALGORITHMS. AVG. = MEAN OVER ALL 10 BENCHMARKS.

Benchmark	AC	PG	QL-MOEA	SoberDSE
nw	0.0	0.0	0.0	0.0
stencil	0.0	0.0	0.0	0.0
atax-medium	**0.0372**	**0.0372**	**0.0372**	**0.0372**
bicg-medium	0.0	0.0	0.0	0.0
correlation	0.0412	0.1625	**0.1641**	0.0412
jacobi-2d	0.0	0.0	0.0	0.0
mvt	0.0	0.0	0.0	0.0
symm	**0.0602**	0.0231	0.0058	0.0058
symm-opt	0.0	0.0	0.0	0.0
syrk	0.0	0.0	0.0	0.0
Avg.	0.0137	0.0223	0.0207	**0.0084**

nature of exploratory algorithms, different runs may produce non-deterministic outcomes. Experimental results in Table VII show that *SoberDSE* achieves the lowest average ADRS, outperforming the SOTA DSE heuristic algorithms Lattice, HGBO-DSE, and MOEDA by 75.79%, 62.83%, and 70.73%, respectively. This demonstrates that the algorithm selection strategy effectively mitigates the limitations of the NFL theorem, enabling *SoberDSE* to deliver superior overall performance across all benchmarks by recommending the potentially optimal algorithm for each case. Additionally, we also compare with SOTA RL Algorithms for DSE including IRONMAN-PRO [19](AC and PG) and QL-MOEA [20]. As shown in Table VIII, darker blue shades indicate better algorithm performance on corresponding benchmarks. Experimental results demonstrate that our proposed *SoberDSE* outperforms SOTA DSE RL algorithms, achieving the best average performance across all benchmarks. Specifically, *SoberDSE* improves over AC by 39.25%, PG by 62.21%, and QL-MOEA by 59.34%. The stable and superior exploration capability of *SoberDSE* confirms the feasibility of using algorithm selection to address DSE optimization challenges.

In summary, based on the evaluation in Sections V-C1-V-C2, our proposed *SoberDSE* method achieves a higher recommendation accuracy than conventional classification models and surpasses SOTA DSE algorithms. These results indicate that addressing DSE challenges need not rely solely on designing new algorithms, but can also be accomplished by recommending suitable algorithms from existing suites based on benchmark characteristics, thereby achieving overall optimal performance. This paradigm shift offers a powerful alternative to designing new algorithms from scratch.

VI. CONCLUSION

This paper presented PortfolioSync-DSE, a dual-adaptive framework that jointly selects kernel-aware multimodal QoR predictors and optimal DSE algorithms to overcome the lack of universal optimality in HLS DSE. Experimental results show that MFF-KAS reduces macro-averaged All RMSE by 30.5% over trimodal fusion and by 39.6% to 67.3% over SOTA predictors, while the underlying trimodal fusion itself already achieves a 13.1% to 52.9% reduction. Integrated into SoberDSE, the framework outperforms state-of-the-art heuristic-based and learning-based DSE methods by up to 69.78% and 53.60%, respectively, while delivering a 50.00% accuracy gain in small-sample scenarios. These results confirm that PortfolioSync-

DSE effectively bridges multimodal representation learning and algorithm portfolio optimization for robust and efficient HLS DSE.

REFERENCES

[1] B. C. Schafer *et al.*, "High-level synthesis design space exploration: Past, present, and future," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 39, no. 10, pp. 2628–2639, 2019.

[2] N. Wu *et al.*, "Ironman: Gnn-assisted design space exploration in high-level synthesis via reinforcement learning," in *GLSVLSI*. ACM, 2021, p. 39–44.

[3] Z. Lin *et al.*, "Powergear: Early-stage power estimation in fpga hls via heterogeneous edge-centric gnns," in *DATE*, 2022, pp. 1341–1346.

[4] H. Kuang *et al.*, "Hgbo-dse: Hierarchical gnn and bayesian optimization based hls design space exploration," in *ICFPT*, 2023, pp. 106–114.

[5] A. Sohrabizadeh *et al.*, "Automated accelerator optimization aided by graph neural networks," in *DAC*. ACM, 2022, p. 55–60.

[6] L. Ferretti *et al.*, "Graph neural networks for high-level synthesis design space exploration," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 28, no. 2, 2022.

[7] M. Gao *et al.*, "Hierarchical source-to-post-route qor prediction in high-level synthesis with gnns," in *DATE*, 2024, pp. 1–6.

[8] B. C. Schafer, "Parallel high-level synthesis design space exploration for behavioral ips of exact latencies," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 22, no. 4, 2017.

[9] B. C. Schafer *et al.*, "Adaptive simulated annealer for high level synthesis design space exploration," in *International Symposium on VLSI Design, Automation and Test*, 2009, pp. 106–109.

[10] B. Carrion Schafer, "Probabilistic multiknob high-level synthesis design space exploration acceleration," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 35, no. 3, pp. 394–406, 2016.

[11] Y. Zhang, S. Wang, and G. Ji, "A comprehensive survey on particle swarm optimization algorithm and its applications," *Mathematical problems in engineering*, vol. 2015, no. 1, p. 931256, 2015.

[12] L. Xu, S. Wang, and C. Xiao, "Mpm-llm4dse: Reaching the pareto frontier in hls with multimodal learning and llm-driven exploration," in *2026 Design, Automation Test in Europe Conference (DATE)*, 2026, pp. 1–7.

[13] Z. Qin *et al.*, "Cross-modality program representation learning for electronic design automation with high-level synthesis," in *MLCAD*. ACM, 2024.

[14] D. H. Wolpert and W. G. Macready, "No free lunch theorems for optimization," *IEEE transactions on evolutionary computation*, vol. 1, no. 1, pp. 67–82, 2002.

[15] L. Xu, S. Wang, and C. Xiao, "Soberdse: Sample-efficient design space exploration via learning-based algorithm selection," in *DAC*. ACM, 2026.

[16] M. Z. S. Khan, N. Mashnoor, M. Akyash, K. Azar, and H. Kamali, "Sage-hls: Syntax-aware ast-guided llm for high-level synthesis code generation," in *2025 IEEE 43rd International Conference on Computer Design (ICCD)*, 2025, pp. 574–581.

[17] L. Ferretti, G. Ansaloni, and L. Pozzi, "Lattice-traversing design space exploration for high level synthesis," in *2018 IEEE 36th International Conference on Computer Design (ICCD)*, 2018, pp. 210–217.

[18] Y. Yao, H. Hong, S. Wang, and C. Xiao, "Decomposition based estimation of distribution algorithm for high-level synthesis design space exploration," *Integration*, vol. 100, p. 102292, 2025.

[19] N. Wu *et al.*, "Ironman-pro: Multiobjective design space exploration in hls via reinforcement learning and graph neural network-based modeling," *IEEE Transactions on Computer-Aided Design of Integrated Circuits and Systems*, vol. 42, no. 3, pp. 900–913, 2023.

[20] Y. Qian, S. Wang, and C. Xiao, "A reinforcement learning based multi-objective evolutionary algorithm for high-level synthesis design space exploration," in *2025 IEEE International Symposium on Circuits and Systems (ISCAS)*. IEEE, 2025, pp. 1–5.

[21] J. D. Owens, M. Houston, D. Luebke, S. Green, J. E. Stone, and J. C. Phillips, "Gpu computing," *Proceedings of the IEEE*, vol. 96, no. 5, pp. 879–899, 2008.

[22] P. Kerschke, H. H. Hoos, F. Neumann, and H. Trautmann, "Automated algorithm selection: Survey and perspectives," *Evolutionary computation*, vol. 27, no. 1, pp. 3–45, 2019.

[23] C. Lattner *et al.*, "Llvm: A compilation framework for lifelong program analysis & transformation," in *International symposium on code generation and optimization*. IEEE, 2004, pp. 75–86.

[24] C. Cummins *et al.*, "Programl: A graph-based program representation for data flow analysis and compiler optimizations," in *ICML*, 2021, pp. 2244–2253.

[25] S. Zhou *et al.*, "CodeBERTScore: Evaluating code generation with pretrained models of code," in *EMNLP*. ACL, 2023, pp. 13 921–13 937.

[26] J. Schulman, F. Wolski, P. Dhariwal, A. Radford, and O. Klimov, "Proximal policy optimization algorithms," *arXiv preprint arXiv:1707.06347*, 2017.

[27] L. Xu, S. Wang, E. Casseau, and C. Xiao, "Ai4dse: Leveraging dynamic graph neural networks and large language models for optimizing high-level synthesis design space exploration," *ACM Trans. Des. Autom. Electron. Syst.*, vol. 31, no. 6, Jun. 2026.

[28] C. Cummins *et al.*, "Programl: A graph-based program representation for data flow analysis and compiler optimizations," in *ICML*, 2021, pp. 2244–2253.

[29] A. Mao, M. Mohri, and Y. Zhong, "Cross-entropy loss functions: Theoretical analysis and applications," in *International conference on Machine learning*. pmlr, 2023, pp. 23 803–23 828.

[30] J. Schulman, P. Moritz, S. Levine, M. Jordan, and P. Abbeel, "High-dimensional continuous control using generalized advantage estimation," *arXiv preprint arXiv:1506.02438*, 2015.

[31] B. Reagen *et al.*, "Machsuite: Benchmarks for accelerator design and customized architectures," in *IISWC*. IEEE, 2014, pp. 110–119.

[32] T. Yuki *et al.*, "Polybenchc, 4.2.1," 2010, https://github.com/MatthiasJReisinger/PolyBenchC-4.2.1.

[33] L. Breiman, "Random forests," *Machine learning*, vol. 45, pp. 5–32, 2001.

[34] T. Chen and C. Guestrin, "Xgboost: A scalable tree boosting system," *Proceedings of the 22nd acm sigkdd international conference on knowledge discovery and data mining*, pp. 785–794, 2016.

[35] T. Cover and P. Hart, "Nearest neighbor pattern classification," *IEEE transactions on information theory*, vol. 13, no. 1, pp. 21–27, 1967.

[36] C. Cortes and V. Vapnik, "Support-vector networks," *Machine learning*, vol. 20, pp. 273–297, 1995.